

TP1 – Secure Log Analyzer in Rust

Module 7.1 – Programming with Rust

Student Name: Taher Ahmedou

Student ID: 21039

GitHub Repository: https://github.com/TAher-Ahmedou/tp1_log_analyzer

Date: May 2026

1. Objective

This tool is a small defensive security utility written in Rust. It reads a simplified Linux authentication log file (SSH daemon logs) and produces a structured summary of suspicious login activity.

The program identifies failed login attempts, extracts the source IP address and targeted username from each event, counts occurrences by IP and by username, and displays the results ranked by frequency. Malformed or unsupported log lines are counted separately and do not interrupt execution.

Why log analysis is useful in defensive security

Authentication logs are one of the most fundamental data sources in defensive security. Analyzing them allows security teams to:

- Detect brute-force attacks: repeated failed logins from the same IP address are a strong indicator of automated credential stuffing or dictionary attacks.
- Identify targeted accounts: accounts that appear frequently in failed login events (such as root, admin, or oracle) are likely high-value targets being probed by attackers.
- Prioritize threat response: by ranking source IPs and targeted usernames, analysts can quickly focus on the most active threats without manually reading thousands of log lines.
- Build baseline behavior: log analysis over time reveals normal patterns, making anomalies easier to detect.

Even a simple tool like this one, applied to real SSH logs, can reveal active scanning campaigns within seconds.

2. Environment

Docker Compose commands used

All development and execution was performed inside the provided Docker Compose environment. The following commands were used:

```
mkdir -p workspace  
docker compose up -d --build  
docker compose exec rustlab bash  
cd /workspace/TPs/tpl_log_analyzer
```

Rust toolchain version

Inside the container, the following toolchain versions were confirmed:

```
rustc 1.86.0 (05f9846f8 2025-03-31)  
cargo 1.86.0 (adf9b6ad1 2025-02-28)
```

Project path inside the container

```
/workspace/TPs/tpl_log_analyzer
```

3. Implementation

3.1 Data model – parser.rs

Two types are defined to represent parsed authentication events:

FailedLogin struct

```
[derive(Debug, Clone, PartialEq, Eq)]
pub struct FailedLogin {
    pub user: String,
    pub ip: String,
}
```

This struct holds the extracted username and source IP address of a failed login attempt. Both fields are owned Strings, which avoids lifetime issues when collecting events from parsed lines.

ParseOutcome enum

```
[derive(Debug, Clone, PartialEq, Eq)]
pub enum ParseOutcome {
    Failed(FailedLogin),
    Ignored,
    Malformed,
}
```

The enum models every possible result of parsing a single log line. This design makes the main loop exhaustive: every line produces exactly one outcome, and the compiler enforces that all three cases are handled.

3.2 Parser function – parse_line()

The parse_line() function classifies a single log line without panicking on unexpected input:

```
pub fn parse_line(line: &str) -> ParseOutcome {
    if line.contains("Accepted password") {
        return ParseOutcome::Ignored;
    }
    if line.contains("Invalid user") && line.contains("from")
        && !line.contains("Failed password") {
        // extract user and ip using and_then
        ...
        return ParseOutcome::Failed(...) or ParseOutcome::Malformed;
    }
    if line.contains("Failed password for") && line.contains("from") {
        // handles both 'for root' and 'for invalid user root'
        ...
        return ParseOutcome::Failed(...) or ParseOutcome::Malformed;
    }
}
```

```
ParseOutcome::Malformed
}
```

The parser handles three distinct SSH log formats:

- Failed password for <user> from <ip> — standard failed login
- Failed password for invalid user <user> from <ip> — login attempt with non-existent username
- Invalid user <user> from <ip> — connection attempt without password phase

All token extraction uses `.and_then()` on `Option` types, which avoids any `unwrap()` and guarantees no panic on malformed input.

3.3 Statistics functions – stats.rs

Two functions aggregate the collected events using `HashMap`:

```
pub fn count_by_ip(events: &[FailedLogin]) -> Vec<(String, usize)> {
    let mut map: HashMap<String, usize> = HashMap::new();
    for event in events {
        *map.entry(event.ip.clone()).or_insert(0) += 1;
    }
    let mut result: Vec<(String, usize)> = map.into_iter().collect();
    result.sort_by(|a, b| b.1.cmp(&a.1));
    result
}
```

`count_by_user()` follows the same pattern using `event.user`. Both functions take a slice (`&[FailedLogin]`), which is idiomatic Rust — they borrow the data without taking ownership. Results are sorted in descending order of count.

3.4 Command-line interface – main.rs

The main function orchestrates the full pipeline:

- Reads the file path from `std::env::args()`
- Prints a usage message and exits cleanly if no argument is provided
- Reads the file content with `fs::read_to_string()`, handling errors with `match` (no `unwrap()`)
- Iterates over all lines, dispatching each to `parse_line()`
- Collects `FailedLogin` events and increments ignored/malformed counters
- Calls `count_by_ip()` and `count_by_user()` on the collected events
- Prints the formatted summary to `stdout`

Error handling example:

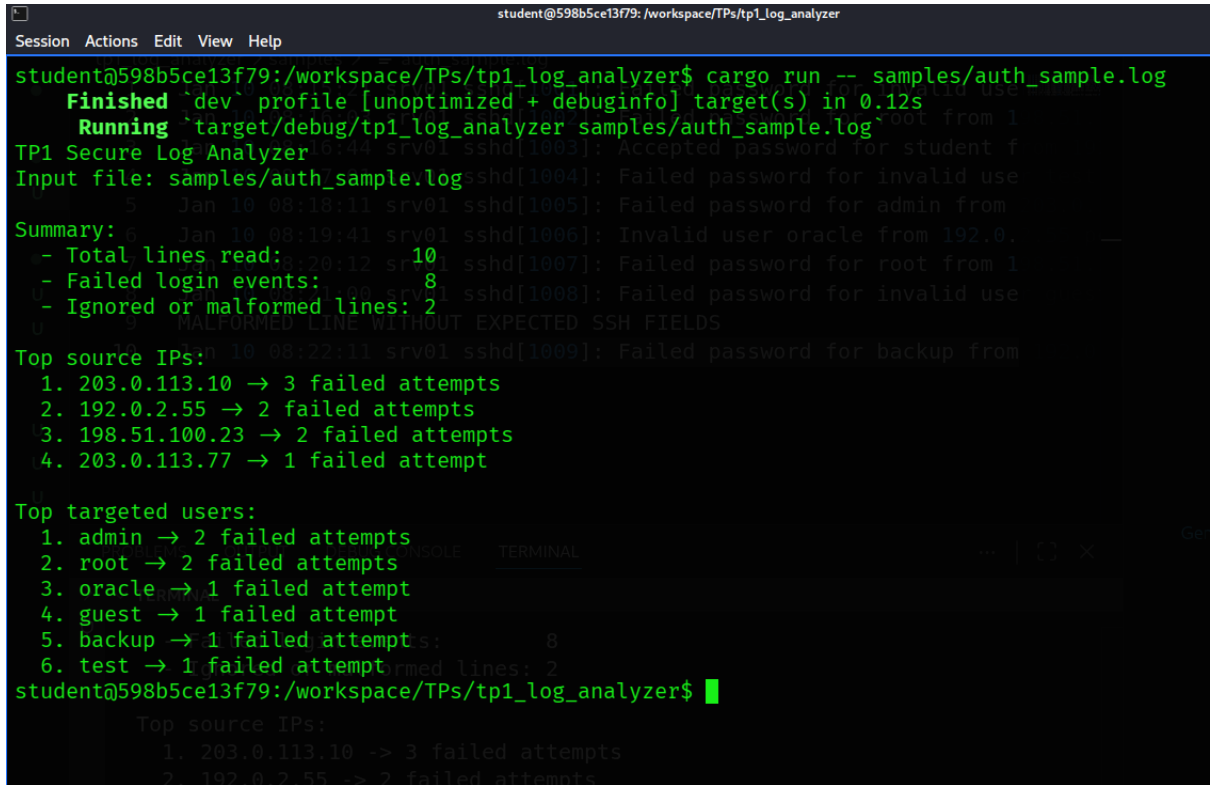
```
let content = match fs::read_to_string(filepath) {
```

```
Ok(c) => c,  
Err(e) => {  
    eprintln!("Error: cannot read file '{}': {}", filepath, e);  
    std::process::exit(1);  
}  
};
```

4. Execution Evidence

The following screenshots demonstrate the correct execution of the program. They are also available in the screenshots/ directory of the GitHub repository.

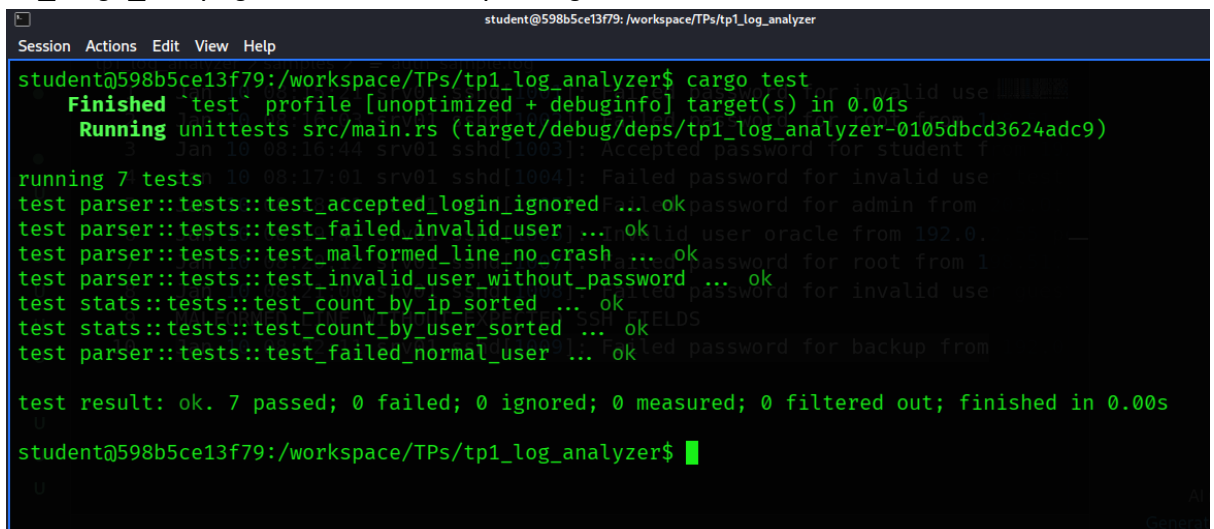
- 01_cargo_run.png — output of cargo run -- samples/auth_sample.log



```
student@598b5ce13f79: /workspace/TPs/tp1_log_analyzer
Session Actions Edit View Help

student@598b5ce13f79:/workspace/TPs/tp1_log_analyzer$ cargo run -- samples/auth_sample.log
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.12s
    Running `target/debug/tp1_log_analyzer samples/auth_sample.log`
TP1 Secure Log Analyzer
Input file: samples/auth_sample.log
Summary:
- Total lines read: 10
- Failed login events: 8
- Ignored or malformed lines: 2
Top source IPs:
1. 203.0.113.10 -> 3 failed attempts
2. 192.0.2.55 -> 2 failed attempts
3. 198.51.100.23 -> 2 failed attempts
4. 203.0.113.77 -> 1 failed attempt
Top targeted users:
1. admin -> 2 failed attempts
2. root -> 2 failed attempts
3. oracle -> 1 failed attempt
4. guest -> 1 failed attempt
5. backup -> 1 failed attempt
6. test -> 1 failed attempt
student@598b5ce13f79:/workspace/TPs/tp1_log_analyzer$
```

- 02_cargo_test.png — all 7 unit tests passing



```
student@598b5ce13f79: /workspace/TPs/tp1_log_analyzer
Session Actions Edit View Help

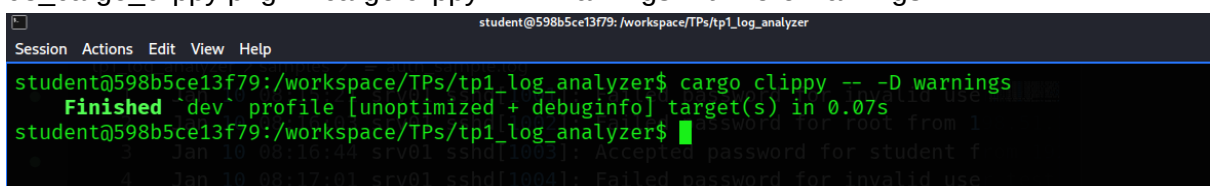
student@598b5ce13f79:/workspace/TPs/tp1_log_analyzer$ cargo test
    Finished `test` profile [unoptimized + debuginfo] target(s) in 0.01s
    Running unittests src/main.rs (target/debug/deps/tp1_log_analyzer-0105dbcd3624adc9)

running 7 tests
test parser::tests::test_accepted_login_ignored ... ok
test parser::tests::test_failed_invalid_user ... ok
test parser::tests::test_malformed_line_no_crash ... ok
test parser::tests::test_invalid_user_without_password ... ok
test stats::tests::test_count_by_ip_sorted ... ok
test stats::tests::test_count_by_user_sorted ... ok
test parser::tests::test_failed_normal_user ... ok

test result: ok. 7 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00s

student@598b5ce13f79:/workspace/TPs/tp1_log_analyzer$
```

- 03_cargo_clippy.png — cargo clippy -- -D warnings with zero warnings



```
student@598b5ce13f79: /workspace/TPs/tp1_log_analyzer
Session Actions Edit View Help

student@598b5ce13f79:/workspace/TPs/tp1_log_analyzer$ cargo clippy -- -D warnings
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.07s
student@598b5ce13f79:/workspace/TPs/tp1_log_analyzer$
```

- 04_expected_output.png — final program output matching the expected format

```

student@598b5ce13f79:/workspace/TPs/tp1_log_analyzer
Session Actions Edit View Help

student@598b5ce13f79:/workspace/TPs/tp1_log_analyzer$ cargo run -- samples/auth_sample.log
Finished dev profile [unoptimized + debuginfo] target(s) in 0.56s
Running target/debug/tp1_log_analyzer samples/auth_sample.log
TP1 Secure Log Analyzer
Input file: samples/auth_sample.log
Jan 10 08:18:11 srv01 sshd[1004]: Failed password for invalid use
5 Jan 10 08:18:11 srv01 sshd[1005]: Failed password for admin from
Summary: 6 Jan 10 08:19:41 srv01 sshd[1006]: Invalid user oracle from 192.0.
- Total lines read: 10
- Failed login events: 8
- Ignored or malformed lines: 2
9 MALFORMED LINE WITHOUT EXPECTED SSH FIELDS
10
Top source IPs:
1. 203.0.113.10 → 3 failed attempts
2. 198.51.100.23 → 2 failed attempts
3. 192.0.2.55 → 2 failed attempts
4. 203.0.113.77 → 1 failed attempt

Top targeted users:
1. root → 2 failed attempts
2. admin → 2 failed attempts
3. backup → 1 failed attempt
4. oracle → 1 failed attempt
5. test → 1 failed attempt
6. guest → 1 failed attempt
student@598b5ce13f79:/workspace/TPs/tp1_log_analyzer$
Top source IPs:
1. 203.0.113.10 -> 3 failed attempts

```

All four commands were executed inside the Docker Compose container at /workspace/TPs/tp1_log_analyzer.

5. Discussion

How malformed lines are handled

The `parse_line()` function returns `ParseOutcome::Malformed` for any line that does not match a known SSH log pattern. This includes:

- Lines with no recognizable keywords (e.g. MALFORMED LINE WITHOUT EXPECTED SSH FIELDS)
- Lines that contain 'Failed password for' but are missing the 'from' keyword
- Lines where word positions are unexpected and token extraction returns `None`

Because all token extraction uses `Option` combinators (`and_then`, `get`), there are no index-out-of-bounds panics. The main loop simply increments the malformed counter and continues to the next line.

Why `Result` and `Option` are useful

`Result` and `Option` are Rust's primary tools for representing fallible operations without using exceptions or silent null values.

`Option<T>` represents a value that may or may not exist. In this program, `parts.get(i + 1)` returns an `Option<&str>`. Using `.and_then()` chains optional lookups safely: if any step returns `None`, the chain short-circuits and returns `None` without panicking.

`Result<T, E>` represents an operation that may succeed or fail with an error. `fs::read_to_string()` returns a `Result`, which forces the caller to handle the error case explicitly. Using `match` instead of `unwrap()` means the program prints a meaningful error message and exits cleanly when the file does not exist.

Both types make failure explicit and visible at the type level, which is one of Rust's core safety guarantees.

What could be improved in a future version

- Add a `--top N` argument to control how many IPs and users are displayed.
- Export results to a CSV file for further analysis in other tools.
- Add timestamp parsing to detect bursts of failed attempts within a short time window.
- Support additional log formats (journald, syslog, etc.) with a configurable parser.
- Add integration tests that read real files from the `samples/` directory and assert on the full output.
- Use a logging library (such as `env_logger`) instead of `eprintln!` for structured error reporting.

6. Conclusion

Summary of what was implemented

This TP produced a complete command-line security tool in Rust that:

- Reads a Linux SSH authentication log file from a command-line argument
- Parses three distinct failed login event formats without crashing on malformed input
- Counts failed attempts by source IP address and by targeted username using HashMap
- Displays a ranked summary of the most suspicious IPs and most targeted users
- Handles all error cases (missing argument, unreadable file) with clean error messages and non-zero exit codes
- Passes cargo fmt, cargo clippy -- -D warnings, and cargo test with 7 unit tests

Main Rust concepts practiced

Concept	Where it was applied
Structs and enums	FailedLogin, ParseOutcome
Option and Result	Token extraction, file reading
Pattern matching	match on ParseOutcome, Result
Ownership and borrowing	&str, &[FailedLogin], .clone()
HashMap	count_by_ip, count_by_user
Iterators	.iter(), .position(), .and_then()
Unit testing	#[cfg(test)], assert_eq!
CLI argument parsing	std::env::args()
Error handling	No unwrap(), no unsafe

This project demonstrated that Rust's type system, when used correctly, makes it possible to write a robust parsing tool that is safe by construction — not just by convention.